

Parallel and Distributed Programming

Introduction

Kenjiro Taura

2026/10/05

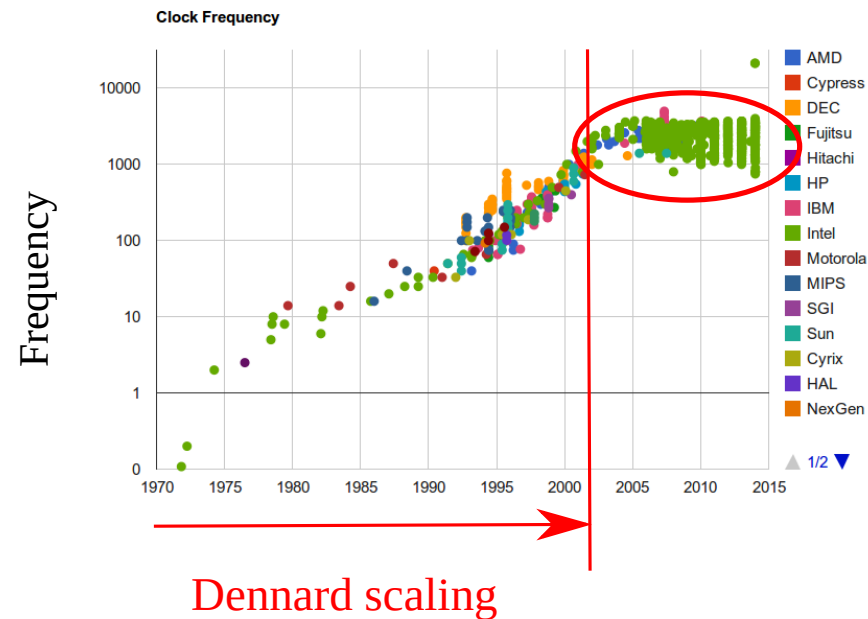
Contents

Why Parallel Programming?	2
What Processors and Parallel Machines Look Like, and Where Performance Come From? ...	4
How to Program Parallel Machines?	11

Why Parallel Programming?

Why parallel?

- frequencies no longer increase (end of Dennard scaling)

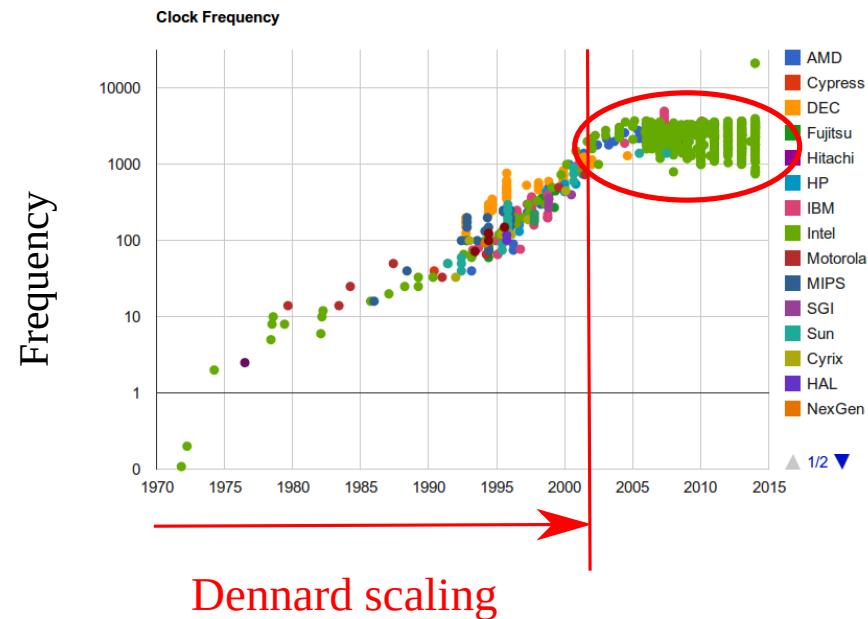


source: cpudb.stanford.edu

There are no *serial* processors any more

Why parallel?

- frequencies no longer increase (end of Dennard scaling)
- techniques to increase performance (Instruction-Level Parallelism, or ILP) of serial programs are increasingly difficult to pay off (Pollack's law)

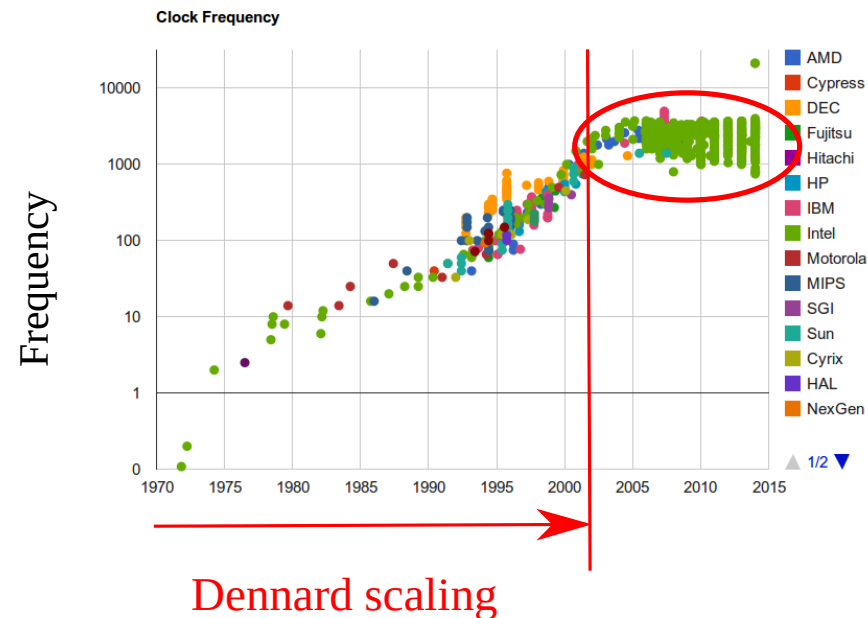


source: cpudb.stanford.edu

There are no *serial* processors any more

Why parallel?

- frequencies no longer increase (end of Dennard scaling)
- techniques to increase performance (Instruction-Level Parallelism, or ILP) of serial programs are increasingly difficult to pay off (Pollack's law)
- multicore, manycore, and GPUs are in part response to it
 - *have more transistors? $\Rightarrow$ have more cores*

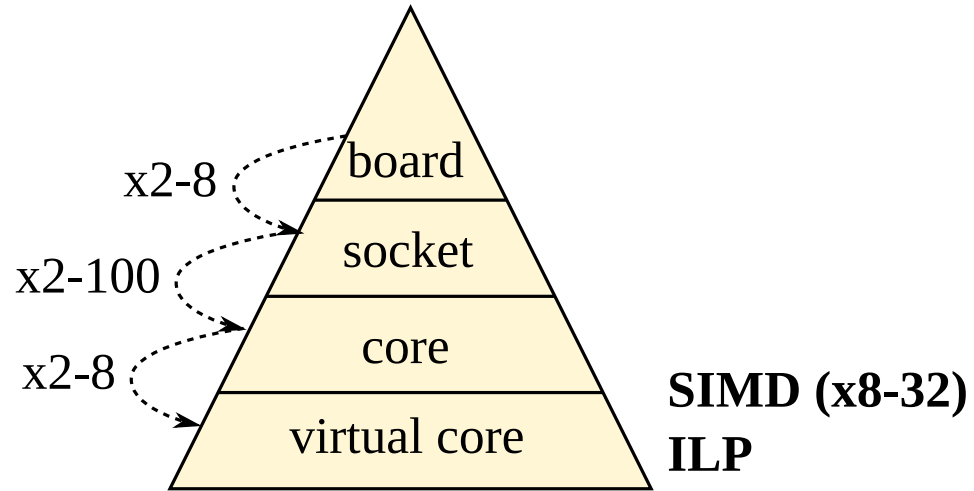
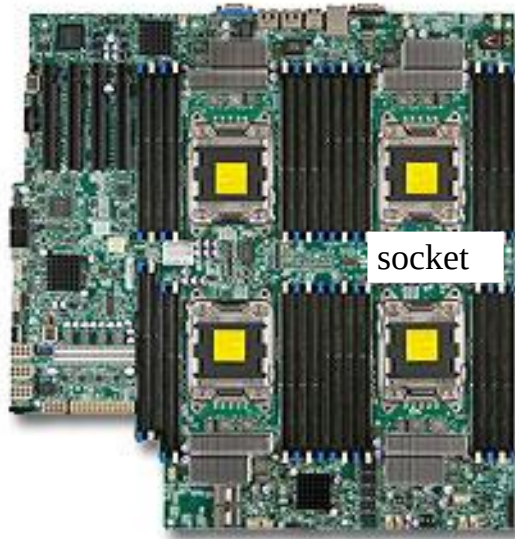


source: cpudb.stanford.edu

There are no *serial* processors any more

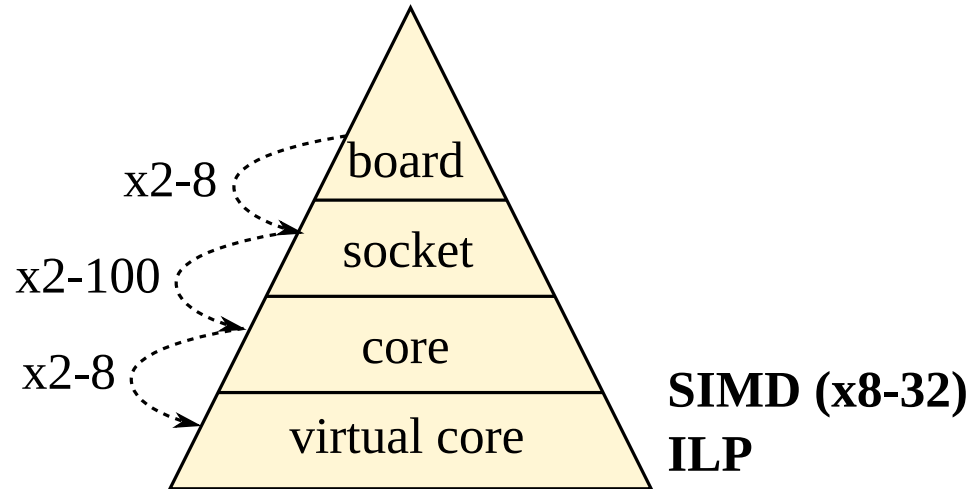
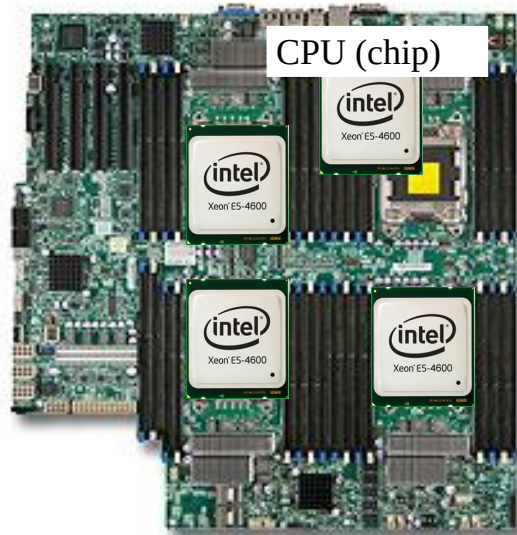
What Processors and Parallel Machines Look Like, and Where Performance Come From?

What a single multicore machine (node) looks like



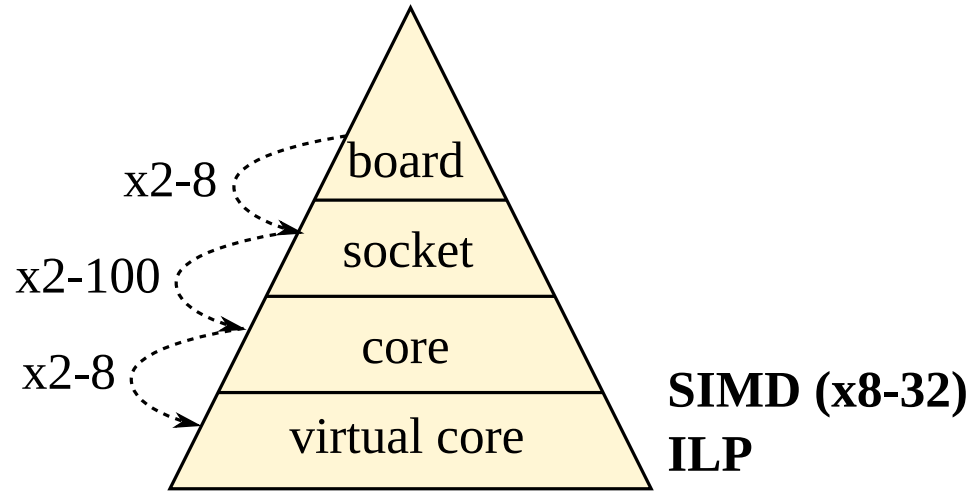
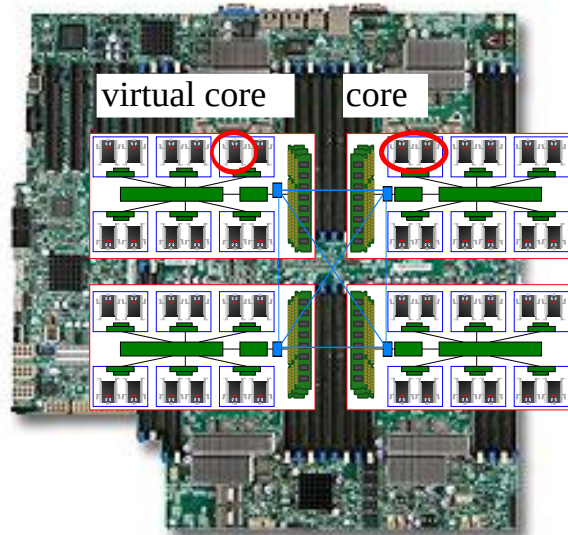
- SIMD : Single Instruction Multiple Data
- ILP : Instruction Level Parallelism

What a single multicore machine (node) looks like



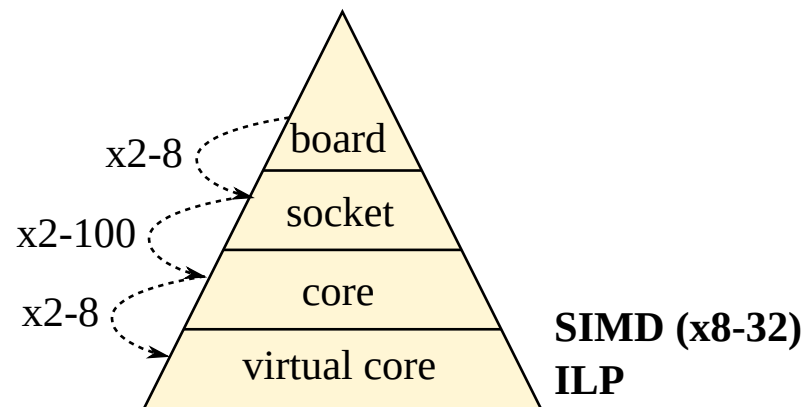
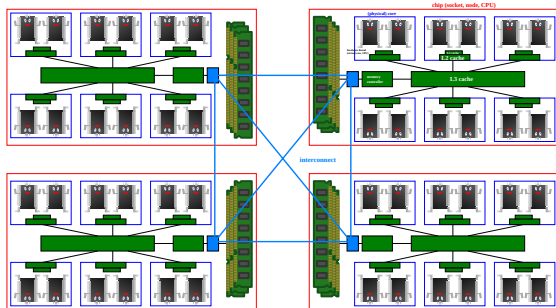
- SIMD : Single Instruction Multiple Data
- ILP : Instruction Level Parallelism

What a single multicore machine (node) looks like



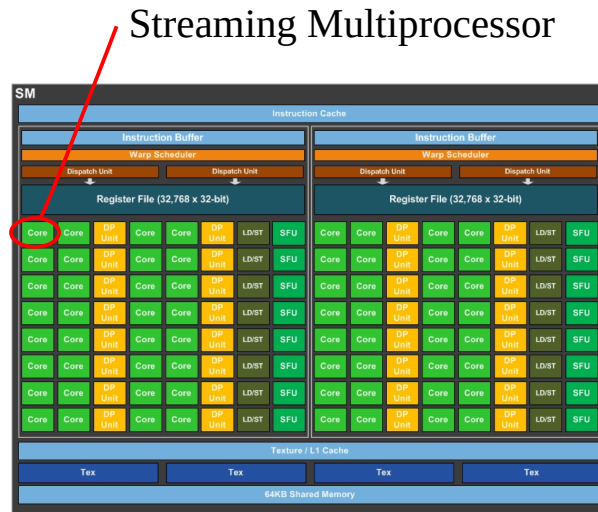
- SIMD : Single Instruction Multiple Data
- ILP : Instruction Level Parallelism

What a machine looks like



- performance comes from *multiplying* parallelism of many levels
- peak performance = SIMD width $\times$ instructions/cycle/core $\times$ freq $\times$ cores
- **peak FLOPS** = $(2 \times \text{SIMD width}) \times \text{FMA insts/cycle/core} \times \text{freq} \times \text{cores}$
- FMA: Fused Multiply Add ($d = a * b + c$)

What a GPU looks like?



- a GPU consists of many *Streaming Multiprocessors (SM)*
- each SM is highly multithreaded and can execute many *warps* simultaneously
- a warp consists of 32 *CUDA threads*, which execute the same single instruction in a cycle
- each CUDA thread has no SIMD nor ILP
- **peak FLOPS** = $(2 \times \text{warp width}) \times \text{warps/cycle/SM} \times \text{freq} \times \text{SMs}$

What a GPU looks like?

- the “big picture” is the same (performance is about multiplying parallelism at all levels)

GPU		CPU
SM	$\approx$	core
multithreading in an SM	$\gg$	simultaneous multithreading
a warp (32 CUDA threads)	$\approx$	a thread executing SIMD instructions
	$\ll$	multiple instructions from a single thread

- differences mainly lie in
 - where parallelism is extracted
 - the way they expose parallelism to the programmer

How much parallelism?

- CPUs

arch model	SIMD width SP/DP	FMA's /cycle/core	freq GHz	core	peak GFLOPS SP/DP	TDP W
Ice Lake 8368	16/8	2	2.4	38	5836/2918	270
Sapphire Rapids 8480+	16/8	2	2.0	56	7168/3584	350
Granite Rapids 6980P	16/8	2	2.0	128	16384/8192	500
Grace (Neoverse V2)	4/2	4	3.1	72	7142/3571	250

- NVIDIA GPUs (without Tensor Cores)

arch model	threads /warp	FMA's /cycle/SM SP/DP	freq GHz	SM	peak GFLOPS SP/DP	TDP W
Volta v100	32	2/1	1.530	80	15667/7833	300
Ampere A100	32	2/1	1.410	108	19353/9676	400
Hopper H100 SXM / GH200	32	4/2	1.98	132	66908/33454	700

Note: NVIDIA Tensor Cores

- performance shown so far is limited by the fact that a single (FMA) instruction can perform 2 flops (1 multiply + 1 add)
- Tensor Core, a special execution unit for a small matrix-multiply-add, changes that
- **H100's each Tensor Core** can do $C = A \times B + C$ (where $A : 4 \times 8$, $B : 8 \times 8$) per cycle (A, B are TF32, C is SP)

$$2 \times 4 \times 8 \times 8 = 512 \text{ flops/cycle}$$

- each SM of H100 GPU has 4 Tensor Cores, so a single H100 (SXM) device can do

$$\begin{aligned} & (2 \times 4 \times 8 \times 8) \text{ [flops/cycle]} \\ & \times \quad \quad \quad 1.83\text{G [cycles/sec]} \\ & \times \quad \quad 4 \times 132 \text{ [Tensor Cores]} \\ & = \quad \quad \quad \mathbf{494700 \text{ GFLOPS}} \end{aligned}$$

- lower precisions go even further: FP16/BF16 989 TFLOPS, FP8 1979 TFLOPS (dense)

How to Program Parallel Machines?

So how to program it?

- no matter how you program it, you want to maximally utilize all forms of parallelism (multicore, multithreading in a single core, SIMD, ILP, ...) and special instructions (e.g., Tensor Cores)
- “how” depends on devices and programming languages

Topics covered

- Programming
 - OpenMP for multicore CPUs
 - SIMD
 - ILP
 - CUDA
 - OpenMP for GPUs
 - Task parallelism
- Foundations
 - What determines performance?
 - latency and throughput
 - FLOPs and memory
 - How to analyze and reason about performance

The course format

- Hands-on experiences on real machines (Miyabi supercomputers, Grace CPU (ARM) + H200 GPU (NVIDIA))
- Programming exercises given in Jupyter notebook
- Course AI provided for questions and generating baseline (details later)
- I will cover fundamental topics but try to leave time for exercises and self-learning through Jupyter notebooks and AIs
- A few weeks later, you choose a “target application” you work on throughout the course
- You apply programming concepts and performance reasoning you learned to the target application
- Lots of in-class work, group work, and frequent mini presentations